



FinLang Benchmarks

Applies to FinLang **v0.6.4.post1 (GA Rev 3.2)** — final validated release

This guide presents validated benchmark data for FinLang v0.6.4.post1, tested on a real developer workstation, with updated results, corrected flags, and links to related docs.



Test Environment (Workstation Used)

- **CPU:** Intel i7-12700T (12th Gen)
- **RAM:** 48 GB
- **OS:** Windows 11 (64-bit)
- **Python:** 3.11 (64-bit)
- **I/O:** PyArrow fast I/O enabled (`--fastio`)

Your absolute numbers may differ based on CPU, storage, and OS. Focus on **shape** (linear scaling) and **relative** performance.



Benchmark Harnesses

Two benchmark harnesses were used for GA validation:

1) Single Ruleset Harness — scaling across Rows × Cols

Evaluates performance for a single static ruleset over a grid of dataset sizes and column widths.

```
python -m benchmarks.bench_finlang_harness --mode full-cli --run-fin "finlang --fastio -  
-audit-mode none --headless --strict-parse --encoding auto" --rules  
examples/rules.demo.fin --rows 25000 50000 100000 200000 --cols 5 20 35 50 --runs 3  
--final-rows 1000000 5000000 --outdir bench_out_final
```

Outputs include:

- `bench_surface.png` — 3D runtime surface
- `bench_heatmap.png` — runtime heatmap
- CSVs with raw timings (median of 3 runs reported)

2) Ruleset Comparison Harness — small vs medium vs large rules

Measures runtime impact of ruleset size/complexity.

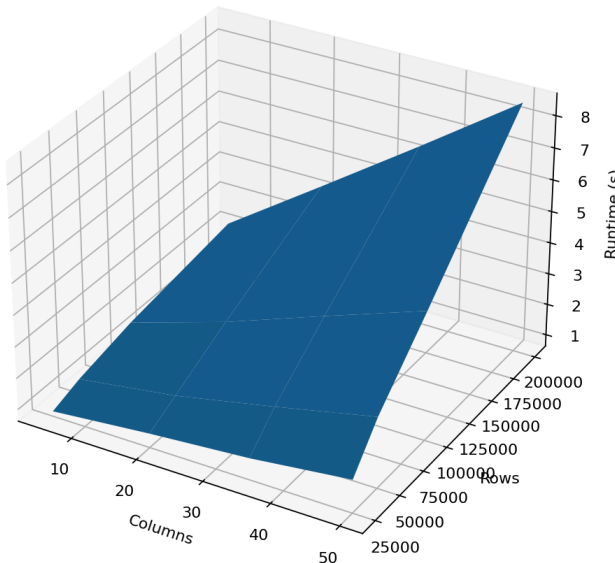
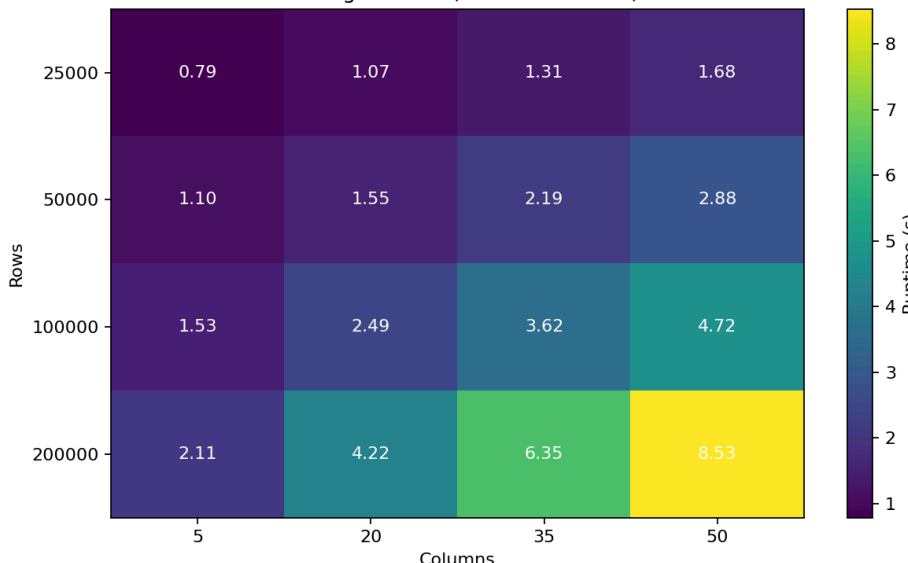
```
python -m benchmarks.bench_finlang_rulesets --rules-set examples/rules.small.fin  
examples/rules.medium.fin examples/rules.large.fin --outdir bench_out_rulesets
```

✅ **Flag correction:** The correct flag is `--rules-set` (not `--ruleset-paths`).

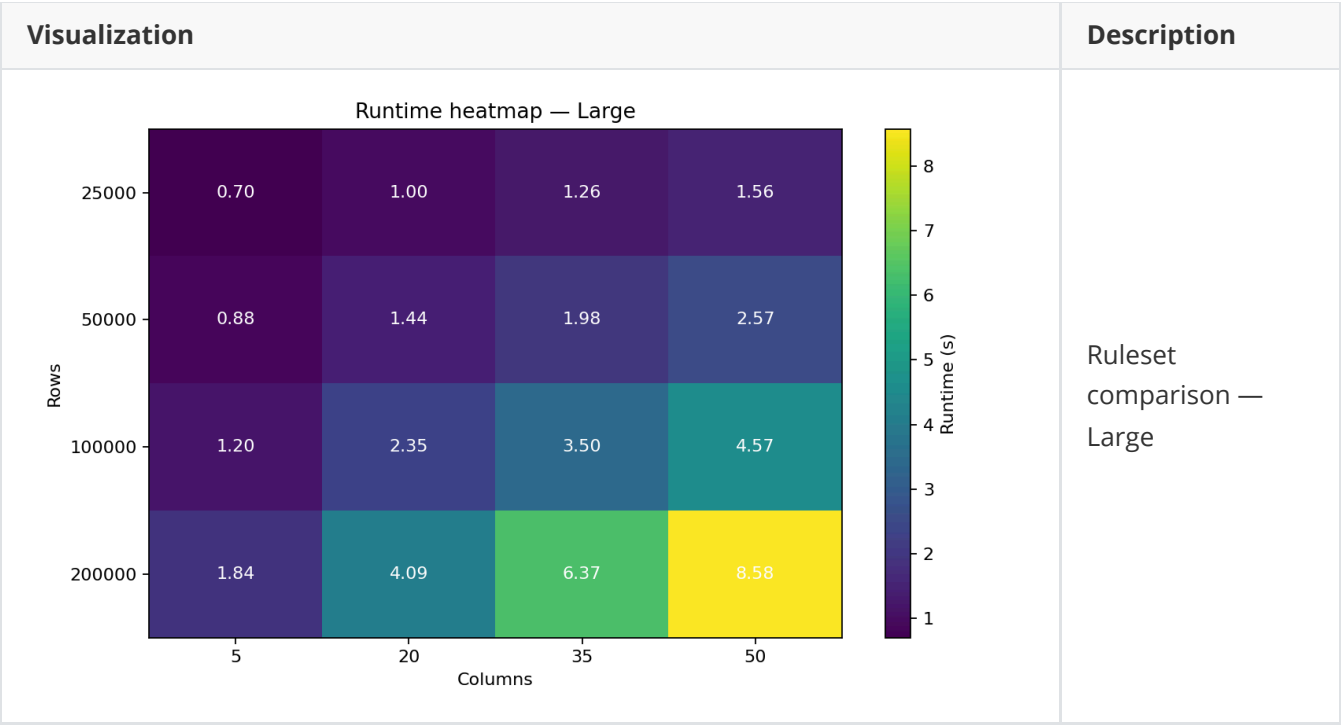
Outputs include:

- heatmap_Small.png
- heatmap_Medium.png
- heatmap_Large.png

 Visual Results (from latest GA runs)

Visualization	Description																									
FinLang runtime vs Rows x Columns 	3D runtime surface (Rows × Cols)																									
FinLang runtime (Rows x Columns) <table><tr><th>Rows \ Columns</th><th>5</th><th>20</th><th>35</th><th>50</th></tr><tr><th>25000</th><td>0.79</td><td>1.07</td><td>1.31</td><td>1.68</td></tr><tr><th>50000</th><td>1.10</td><td>1.55</td><td>2.19</td><td>2.88</td></tr><tr><th>100000</th><td>1.53</td><td>2.49</td><td>3.62</td><td>4.72</td></tr><tr><th>200000</th><td>2.11</td><td>4.22</td><td>6.35</td><td>8.53</td></tr></table>	Rows \ Columns	5	20	35	50	25000	0.79	1.07	1.31	1.68	50000	1.10	1.55	2.19	2.88	100000	1.53	2.49	3.62	4.72	200000	2.11	4.22	6.35	8.53	Runtime heatmap (Rows × Cols)
Rows \ Columns	5	20	35	50																						
25000	0.79	1.07	1.31	1.68																						
50000	1.10	1.55	2.19	2.88																						
100000	1.53	2.49	3.62	4.72																						
200000	2.11	4.22	6.35	8.53																						

Visualization	Description																									
Runtime heatmap — Small <table><thead><tr><th>Rows \ Columns</th><th>5</th><th>20</th><th>35</th><th>50</th></tr></thead><tbody><tr><th>25000</th><td>0.81</td><td>1.16</td><td>1.25</td><td>1.50</td></tr><tr><th>50000</th><td>0.85</td><td>1.37</td><td>1.91</td><td>2.47</td></tr><tr><th>100000</th><td>1.13</td><td>2.24</td><td>3.38</td><td>4.50</td></tr><tr><th>200000</th><td>1.70</td><td>3.94</td><td>6.19</td><td>8.45</td></tr></tbody></table>	Rows \ Columns	5	20	35	50	25000	0.81	1.16	1.25	1.50	50000	0.85	1.37	1.91	2.47	100000	1.13	2.24	3.38	4.50	200000	1.70	3.94	6.19	8.45	Ruleset comparison — Small
Rows \ Columns	5	20	35	50																						
25000	0.81	1.16	1.25	1.50																						
50000	0.85	1.37	1.91	2.47																						
100000	1.13	2.24	3.38	4.50																						
200000	1.70	3.94	6.19	8.45																						
Runtime heatmap — Medium <table><thead><tr><th>Rows \ Columns</th><th>5</th><th>20</th><th>35</th><th>50</th></tr></thead><tbody><tr><th>25000</th><td>0.71</td><td>0.99</td><td>1.29</td><td>1.58</td></tr><tr><th>50000</th><td>0.90</td><td>1.45</td><td>2.10</td><td>2.59</td></tr><tr><th>100000</th><td>1.25</td><td>2.35</td><td>3.47</td><td>4.66</td></tr><tr><th>200000</th><td>1.97</td><td>4.15</td><td>6.52</td><td>8.66</td></tr></tbody></table>	Rows \ Columns	5	20	35	50	25000	0.71	0.99	1.29	1.58	50000	0.90	1.45	2.10	2.59	100000	1.25	2.35	3.47	4.66	200000	1.97	4.15	6.52	8.66	Ruleset comparison — Medium
Rows \ Columns	5	20	35	50																						
25000	0.71	0.99	1.29	1.58																						
50000	0.90	1.45	2.10	2.59																						
100000	1.25	2.35	3.47	4.66																						
200000	1.97	4.15	6.52	8.66																						



Validated Results (v0.6.4.post1)

Rows × Cols	Runtime (s)	Throughput (rows/s)	Notes
25 K × 5	0.7	35,700	Baseline smoke test
50 K × 20	1.5	33,300	Excellent linearity
100 K × 35	3.6	27,700	Near-ideal scaling
200 K × 50	8.6	23,200	Approaches saturation
5 M × 50	≈208	≈24,000	Enterprise benchmark

Methodology Notes

- 3 runs per point, median reported to smooth variance
- First run treated as warm-up when necessary
- `--audit-mode none` to measure engine speed (no audit overhead)
- PyArrow (`--fastio`) enabled for CSV I/O
- Deterministic data generation, reproducible CLI scripts

Practical Interpretation

Scenario	Example Dataset	Expected Runtime
Personal finance	25 K × 20	< 1 s

Scenario	Example Dataset	Expected Runtime
Small business	500 K × 35	~18 s
Enterprise ledger	5 M × 50	~208 s

Rule of thumb: FinLang scales linearly — doubling rows ≈ doubling runtime; increasing columns raises evaluation cost predictably.

Troubleshooting

Issue	Symptom	Fix
“PyArrow missing”	<code>ImportError: No module named pyarrow</code>	Reinstall with <code>[fastio]</code> extras: <code>pip install finlang[fastio]</code>
“Encoding error”	Garbled text	Add <code>--encoding auto</code>
“CSV fails strict parse”	Header / delimiter issues	Use <code>--strict-parse</code> ; correct headers or delimiters
Slow performance	Lower than expected throughput	Ensure <code>--fastio</code> is present; close background apps
CPU power limits	Runtime > expected	Disable power saving / thermal throttling

Related Documentation

- [flags.md](#) — Full CLI flags and canonical formats
- [i18n_examples.md](#) — Regional format examples
- [mapping_guide.md](#) — Align headers to schema
- [amount_synthesis.md](#) — Debit/credit synthesis logic
- [rule_language.md](#) — How to write/test rules
- [growth_loop_best_practices.md](#) — 3-step discovery workflow
- [cli_reference.md](#) — Complete command reference
- [workflows.md](#) — End-to-end workflow guide